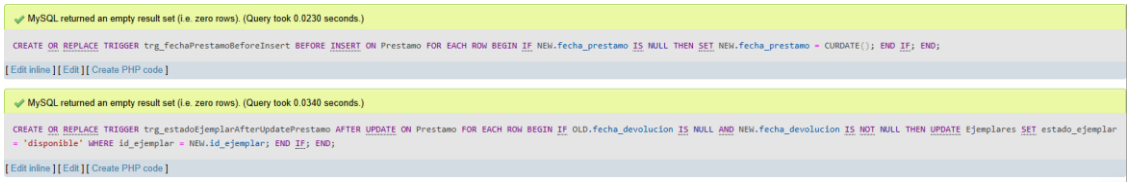
Punto 1. Definición de 2 disparadores/triggers sobre operaciones asociadas al modelo de datos.

```sql
DELIMITER $$
CREATE OR REPLACE TRIGGER trg_fechaPrestamoBeforeInsert
BEFORE INSERT ON Prestamo FOR EACH ROW
BEGIN
    IF NEW.fecha_prestamo IS NULL THEN
        SET NEW.fecha_prestamo = CURDATE();
    END IF;
END $$
DELIMITER ;
```

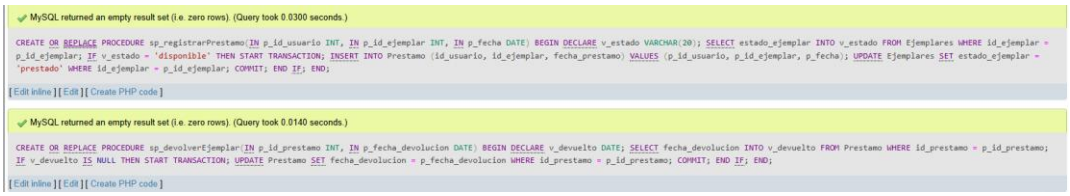
```sql
DELIMITER $$
CREATE OR REPLACE TRIGGER trg_estadoEjemplarAfterUpdatePrestamo
AFTER UPDATE ON Prestamo FOR EACH ROW
BEGIN
    IF OLD.fecha_devolucion IS NULL AND NEW.fecha_devolucion IS NOT NULL THEN
        UPDATE Ejemplares SET estado_ejemplar = 'disponible' WHERE id_ejemplar = NEW.id_ejemplar;
    END IF;
END $$
DELIMITER ;
```



Punto 2. Definición de 2 procedimientos almacenados que realicen más de una operación dentro de una transacción, haciendo una gestión adecuada de los errores (mediante mensajes).

```sql
DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_registrarPrestamo(IN p_id_usuario INT, IN p_id_ejemplar INT, IN p_fecha DATE)
BEGIN
    DECLARE v_estado VARCHAR(20);
    SELECT estado_ejemplar INTO v_estado FROM Ejemplares WHERE id_ejemplar = p_id_ejemplar;
    IF v_estado = 'disponible' THEN
        START TRANSACTION;
        INSERT INTO Prestamo (id_usuario, id_ejemplar, fecha_prestamo) VALUES (p_id_usuario, p_id_ejemplar, p_fecha);
        UPDATE Ejemplares SET estado_ejemplar = 'prestado' WHERE id_ejemplar = p_id_ejemplar;
        COMMIT;
    END IF;
END $$
DELIMITER ;
```

```sql
DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_devolverEjemplar(IN p_id_prestamo INT, IN p_fecha_devolucion DATE)
BEGIN
    DECLARE v_devuelto DATE;
    SELECT fecha_devolucion INTO v_devuelto FROM Prestamo WHERE id_prestamo = p_id_prestamo;
    IF v_devuelto IS NULL THEN
        START TRANSACTION;
        UPDATE Prestamo SET fecha_devolucion = p_fecha_devolucion WHERE id_prestamo = p_id_prestamo;
        COMMIT;
    END IF;
END $$
DELIMITER ;
```



Punto 3. Definición de 2 procedimientos almacenados que utilicen cursores que recorran cierta cantidad de datos, realizando operaciones sobre una o más tablas, haciendo una gestión adecuada de los errores (mediante mensajes).

```sql
CREATE TABLE IF NOT EXISTS log_prestamos_activos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_prestamo INT,
    nombre_usuario VARCHAR(100),
    titulo_libro VARCHAR(200),
    fecha_prestamo DATE,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS log_mantenimiento (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_ejemplar INT,
    copia_ejemplar VARCHAR(50),
    isbn VARCHAR(20),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_registrarPrestamosActivos()
BEGIN
    DECLARE v_id_prestamo INT; DECLARE v_nombre VARCHAR(100); DECLARE v_titulo VARCHAR(200);
    DECLARE v_fecha DATE; DECLARE fin INT DEFAULT 0;
    DECLARE cur CURSOR FOR
        SELECT P.id_prestamo, U.nombre, L.titulo, P.fecha_prestamo
        FROM Prestamo P
        JOIN Usuarios U ON P.id_usuario = U.id_usuario
        JOIN Ejemplares E ON P.id_ejemplar = E.id_ejemplar
        JOIN Libros L ON E.isbn = L.isbn
        WHERE P.fecha_devolucion IS NULL;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;
    DELETE FROM log_prestamos_activos;
    OPEN cur;
    bucle: LOOP
        FETCH cur INTO v_id_prestamo, v_nombre, v_titulo, v_fecha;
        IF fin = 1 THEN LEAVE bucle; END IF;
        INSERT INTO log_prestamos_activos (id_prestamo, nombre_usuario, titulo_libro, fecha_prestamo)
        VALUES (v_id_prestamo, v_nombre, v_titulo, v_fecha);
    END LOOP bucle;
    CLOSE cur;
END $$
DELIMITER ;

DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_liberarEjemplaresMantenimiento()
BEGIN
    DECLARE v_id INT; DECLARE v_copia VARCHAR(50); DECLARE v_isbn VARCHAR(20); DECLARE fin INT DEFAULT 0;
    DECLARE cur CURSOR FOR SELECT id_ejemplar, copia_ejemplar, isbn FROM Ejemplares WHERE estado_ejemplar = 'mantenimiento';
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;
    OPEN cur;
```

```
    bucle: LOOP
        FETCH cur INTO v_id, v_copia, v_isbn;
        IF fin = 1 THEN LEAVE bucle; END IF;
        INSERT INTO log_mantenimiento (id_ejemplar, copia_ejemplar, isbn) VALUES (v_id, v_copia, v_isbn);
        UPDATE Ejemplares SET estado_ejemplar = 'disponible' WHERE id_ejemplar = v_id;
    END LOOP bucle;
    CLOSE cur;
END $$
DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0160 seconds.)

```
CREATE TABLE IF NOT EXISTS log_prestamos_activos ( id INT AUTO_INCREMENT PRIMARY KEY, id_prestamo INT, nombre_usuario VARCHAR(100), titulo_libro VARCHAR(200), fecha_prestamo DATE, fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[Edit inline] [Edit] [Create PHP code]

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0160 seconds.)

```
CREATE TABLE IF NOT EXISTS log_mantenimiento ( id INT AUTO_INCREMENT PRIMARY KEY, id_ejemplar INT, copia_ejemplar VARCHAR(50), isbn VARCHAR(20), fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[Edit inline] [Edit] [Create PHP code]

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0180 seconds.)

```
CREATE OR REPLACE PROCEDURE sp_registrarPrestamosActivos() BEGIN DECLARE v_id_prestamo INT; DECLARE v_nombre VARCHAR(100); DECLARE v_titulo VARCHAR(200); DECLARE v_fecha DATE; DECLARE fin INT DEFAULT 0; DECLARE cur CURSOR FOR SELECT P.id_prestamo, U.nombre, L.titulo, P.fecha_prestamo FROM Prestamo P JOIN Usuarios U ON P.id_usuario = U.id_usuario JOIN Ejemplares E ON P.id_ejemplar = E.id_ejemplar JOIN Libros L ON E.isbn = L.isbn WHERE P.fecha_devolucion IS NULL; DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1; DELETE FROM log_prestamos_activos; OPEN cur; bucle: LOOP FETCH cur INTO v_id_prestamo, v_nombre, v_titulo, v_fecha; IF fin = 1 THEN LEAVE bucle; END IF; INSERT INTO log_prestamos_activos (id_prestamo, nombre_usuario, titulo_libro, fecha_prestamo) VALUES (v_id_prestamo, v_nombre, v_titulo, v_fecha); END LOOP bucle; CLOSE cur; END;
```

[Edit inline] [Edit] [Create PHP code]

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0080 seconds.)

```
CREATE OR REPLACE PROCEDURE sp_liberarEjemplaresMantenimiento() BEGIN DECLARE v_id INT; DECLARE v_copia VARCHAR(50); DECLARE v_isbn VARCHAR(20); DECLARE fin INT DEFAULT 0; DECLARE cur CURSOR FOR SELECT id_ejemplar, copia_ejemplar, isbn FROM Ejemplares WHERE estado_ejemplar = 'mantenimiento'; DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1; OPEN cur; bucle: LOOP FETCH cur INTO v_id, v_copia, v_isbn; IF fin = 1 THEN LEAVE bucle; END IF; INSERT INTO log_mantenimiento (id_ejemplar, copia_ejemplar, isbn) VALUES (v_id, v_copia, v_isbn); UPDATE Ejemplares SET estado_ejemplar = 'disponible' WHERE id_ejemplar = v_id; END LOOP bucle; CLOSE cur; END;
```

[Edit inline] [Edit] [Create PHP code]

Punto 4. Creación de 2 funciones que consulten datos de varias tablas y con información relevante para la explotación de información.

```
DROP FUNCTION IF EXISTS fn_totalPrestamosUsuario;
DROP FUNCTION IF EXISTS fn_autorLibro;
DELIMITER $$
CREATE FUNCTION fn_totalPrestamosUsuario(p_id_usuario INT) RETURNS INT DETERMINISTIC
BEGIN
    DECLARE v_total INT;
    SELECT COUNT(*) INTO v_total FROM Prestamo WHERE id_usuario = p_id_usuario;
    RETURN v_total;
END $$
DELIMITER ;
```

```
DELIMITER $$
CREATE FUNCTION fn_autorLibro(p_isbn VARCHAR(20)) RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    DECLARE v_nombre VARCHAR(100);
    SELECT A.nombre_autor INTO v_nombre FROM Autor A JOIN es_escrito E ON A.id_autor = E.id_autor WHERE E.isbn = p_isbn LIMIT 1;
    RETURN COALESCE(v_nombre, 'Autor desconocido');
END $$
DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0070 seconds.)

```
DROP FUNCTION IF EXISTS fn_totalPrestamosUsuario;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Note: #1305 FUNCTION biblioteca_bd.fn_totalPrestamosUsuario does not exist

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0020 seconds.)

```
DROP FUNCTION IF EXISTS fn_autorLibro;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Note: #1305 FUNCTION biblioteca_bd.fn_autorLibro does not exist

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0080 seconds.)

```
CREATE FUNCTION fn_totalPrestamosUsuario(p_id_usuario INT) RETURNS INT DETERMINISTIC BEGIN DECLARE v_total INT; SELECT COUNT(*) INTO v_total FROM Prestamo WHERE id_usuario = p_id_usuario; RETURN v_total; END;
```

[Edit inline] [Edit] [Create PHP code]

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0090 seconds.)

```
CREATE FUNCTION fn_autorLibro(p_isbn VARCHAR(20)) RETURNS VARCHAR(100) DETERMINISTIC BEGIN DECLARE v_nombre VARCHAR(100); SELECT A.nombre_autor INTO v_nombre FROM Autor A JOIN es_escrito E ON A.id_autor = E.id_autor WHERE E.isbn = p_isbn LIMIT 1; RETURN COALESCE(v_nombre, 'Autor desconocido'); END;
```

[Edit inline] [Edit] [Create PHP code]

Punto 5. Creación de un script que haga uso de un conjunto de procedimientos/funciones que colaboren entre sí para generar un informe estadístico de la BD de vuestro sistema de información. Apóyate en una tabla dashboard donde se refleje la información recopilada y que posteriormente sea consultada.

```sql
-- DASHBOARD
CREATE TABLE IF NOT EXISTS dashboard_estadisticas (
    id INT AUTO_INCREMENT PRIMARY KEY, indicador VARCHAR(150) NOT NULL,
    valor_numerico INT DEFAULT 0, valor_texto VARCHAR(200) DEFAULT NULL,
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);

DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_generarDashboard()
BEGIN
    CALL sp_registrarPrestamosActivos();
    CALL sp_liberarEjemplaresMantenimiento();
    DELETE FROM dashboard_estadisticas;
    INSERT INTO dashboard_estadisticas (indicador, valor_numerico)
        SELECT CONCAT('Préstamos del usuario: ', nombre), fn_totalPrestamosUsuario(id_usuario) FROM Usuarios;
    INSERT INTO dashboard_estadisticas (indicador, valor_numerico) VALUES
        ('Total libros en catálogo',(SELECT COUNT(*) FROM Libros)),
        ('Total usuarios registrados',(SELECT COUNT(*) FROM Usuarios)),
        ('Total ejemplares',(SELECT COUNT(*) FROM Ejemplares)),
        ('Ejemplares disponibles',(SELECT COUNT(*) FROM Ejemplares WHERE estado_ejemplar = 'disponible')),
        ('Préstamos activos',(SELECT COUNT(*) FROM Prestamo WHERE fecha_devolucion IS NULL)),
        ('Préstamos totales (histórico)',(SELECT COUNT(*) FROM Prestamo));
    INSERT INTO dashboard_estadisticas (indicador, valor_texto)
        SELECT 'Libro más prestado', CONCAT(L.titulo, ' (', fn_autorLibro(L.isbn), ')')
        FROM Libros L JOIN Ejemplares E ON L.isbn = E.isbn JOIN Prestamo P ON E.id_ejemplar = P.id_ejemplar
        GROUP BY L.isbn, L.titulo ORDER BY COUNT(*) DESC LIMIT 1;
    SELECT 'OK: Dashboard generado correctamente.' AS mensaje;
END $$
DELIMITER ;

-- EJECUCIÓN
CALL sp_generarDashboard();
SELECT indicador, valor_numerico, valor_texto, fecha_actualizacion FROM dashboard_estadisticas;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0140 seconds.)

```
-- DASHBOARD CREATE TABLE IF NOT EXISTS dashboard_estadisticas ( id INT AUTO_INCREMENT PRIMARY KEY, indicador VARCHAR(150) NOT NULL, valor_numerico INT DEFAULT 0, valor_texto VARCHAR(200) DEFAULT NULL,
fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);
```

[Edit inline] [Edit] [Create PHP code]

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0130 seconds.)

```
CREATE OR REPLACE PROCEDURE sp_generarDashboard() BEGIN CALL sp_registrarPrestamosActivos(); CALL sp_liberarEjemplaresMantenimiento(); DELETE FROM dashboard_estadisticas; INSERT INTO dashboard_estadisticas (indicador,
valor_numerico) SELECT CONCAT('Préstamos del usuario: ', nombre), fn_totalPrestamosUsuario(id_usuario) FROM Usuarios; INSERT INTO dashboard_estadisticas (indicador, valor_numerico) VALUES ('Total libros en catálogo',
(SELECT COUNT(*) FROM Libros)), ('Total usuarios registrados',(SELECT COUNT(*) FROM Usuarios)), ('Total ejemplares',(SELECT COUNT(*) FROM Ejemplares)), ('Ejemplares disponibles',(SELECT COUNT(*) FROM Ejemplares WHERE
estado_ejemplar = 'disponible')), ('Préstamos activos',(SELECT COUNT(*) FROM Prestamo WHERE fecha_devolucion IS NULL)), ('Préstamos totales (histórico)',(SELECT COUNT(*) FROM Prestamo)); INSERT INTO
dashboard_estadisticas (indicador, valor_texto) [...]
```

[Edit]

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Showing rows 0 - 0 (1 total, Query took 0.1570 seconds.)

```
-- EJECUCIÓN CALL sp_generarDashboard();
```

[Edit inline] [Edit] [Create PHP code]

Showing rows 0 - 11 (12 total, Query took 0.0040 seconds.)

```
SELECT indicador, valor_numerico, valor_texto, fecha_actualizacion FROM dashboard_estadisticas;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	indicador	valor_numerico	valor_texto	fecha_actualizacion
Edit Copy Delete	Préstamos del usuario: Juan Pérez	1	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos del usuario: María González	1	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos del usuario: Carlos Rodríguez	1	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos del usuario: Ana Martínez	0	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos del usuario: Luis Fernández	0	NULL	2026-05-22 19:56:26
Edit Copy Delete	Total libros en catálogo	5	NULL	2026-05-22 19:56:26
Edit Copy Delete	Total usuarios registrados	5	NULL	2026-05-22 19:56:26
Edit Copy Delete	Total ejemplares	7	NULL	2026-05-22 19:56:26
Edit Copy Delete	Ejemplares disponibles	6	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos activos	0	NULL	2026-05-22 19:56:26
Edit Copy Delete	Préstamos totales (histórico)	3	NULL	2026-05-22 19:56:26
Edit Copy Delete	Libro más prestado	0	La casa de los espíritus (Isabel Allende)	2026-05-22 19:56:26

Check all With selected: Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None